Instituto Tecnológico y de Estudios Superiores de Monterrey

A Lightweight Model for Brain MRI Segmentation

Jesus Ramirez Delgado

Development of Advanced Computer Science Applications

Abstract

This project presents the development and evaluation of lightweight deep learning models for brain MRI segmentation using the LGG Segmentation Dataset. The task focuses on binary segmentation of FLAIR abnormality regions from 2D MRI slices with a strong class imbalance, where lesion pixels represent only 1.03% of the dataset. Several approaches were tested, starting with a threshold-based baseline and progressing through multiple encoder-decoder architectures inspired by state-of-the-art segmentation models. The models were evaluated using 5-fold cross-validation and a separate final test set. The final model, encoder-decoder v4, combines skip connections, separable convolutions, MaxPooling, BCE + Dice Loss, and a deeper but lightweight encoder-decoder structure. The results show that V4 achieved a strong balance between segmentation performance and computational efficiency, reaching a global cross-validation IoU of 0.520367 and a final test IoU of 0.600055, while reducing training time to approximately 3 hours. These results indicate that the proposed lightweight architecture can provide competitive segmentation performance while significantly reducing computational cost.

Index

1. Introduction **3**
1.1 Objective 3
2. Data Exploration **3**
2.1 Dataset Introduction 3
2.2 Structure of the Dataset 4
2.3 File Types and Format 4
2.4 Number of Patients and Samples (instances) 4
2.5 Additional Information 5
2.6 Exploration 5
2.6.1 Average Image 5
2.6.2 Class distribution 6
2.7 Data quality 7
3.0 Data Preparation **7**
3.1 Data split 7
3.2 Data augmentation 8
4.0 Modeling **9**
4.1 Threshold-based segmentation 10
4.1.1 Parameters and configuration 10
4.2 Encoder - decoder v1 11
4.2.1 Parameters and configuration 11
4.2.2 Architecture 12
4.3 Encoder - decoder v2 13
4.3.1 Changes and improvements 13
4.3.2 Parameters and configuration 14
4.3.3 Architecture 16

4.4 Encoder - decoder v3 16
4.4.1 Changes and improvements 16
4.4.2 Parameters and configuration 17
4.4.3 Architecture 18
4.5 Encoder - decoder v4 18
4.5.1 Changes and improvements 19
4.5.2 Parameters and configuration 20
4.5.3 Architecture 20
5.0 Evaluation 21
5.1 Threshold-based segmentation 21
5.1.1 Results 21
5.1.2 Metrics analysis 22
5.1.3 Folds and test analysis 22
5.1.4 Conclusions 23
5.2 Encoder - decoder V1 23
5.2.1 Results 23
5.2.2 Metrics analysis 23
5.2.3 Folds and test analysis 24
5.2.4 Conclusions 24
5.3 Encoder - decoder V2 24
5.3.1 Results 24
5.3.4 Conclusions 24
5.4 Encoder - decoder V3 25
5.4.1 Results 25
5.4.2 Metrics analysis 25
5.4.3 Fold and test analysis 26
5.4.4 Conclusions 26
5.5 Encoder - decoder V4 27
5.4.1 Results 27
5.5.2 Metrics Analysis 27
5.5.3 Folds and test analysis 28
5.5.4 Conclusión 28
6.0 Model comparison 28
7.0 Predictions 29
8.0 General Conclusions 31
References 32

1. Introduction

Brain MRI segmentation is an important task in medical image analysis because it allows abnormal regions to be identified at the pixel level. In this project, the objective is to segment lower-grade glioma abnormality regions using MRI images from the LGG Segmentation Dataset. Each MRI image has a size of $256 \times 256 \times 3$, while each corresponding mask is a binary image that identifies the abnormal region. The dataset contains 110 patients, 3,929 MRI images, and 3,929 segmentation masks.

One of the main challenges of this task is the strong class imbalance between background and lesion pixels. Background pixels represent 98.97% of the dataset, while lesion pixels represent only 1.03%. Because of this, simple accuracy is not enough to evaluate model performance, and metrics such as IoU, Precision, and Recall are more informative.

This project evaluates different segmentation approaches, beginning with a threshold-based baseline and then moving to encoder-decoder neural network architectures. Through several iterations, the models were refined to compare segmentation quality, stability, and computational efficiency.

1.1 Objective

The objective of this work was to obtain strong segmentation results for brain MRI abnormality regions while developing a lightweight model inspired by state-of-the-art segmentation architectures. The project first established a simple threshold-based segmentation model as a non-machine-learning and non-deep-learning reference baseline for comparison. Then, a first encoder-decoder architecture was developed as the main base model to beat in terms of segmentation performance and computational cost.

2. Data Exploration

2.1 Dataset Introduction

The dataset source comes from "*Brain MRI segmentation*" by Mateusz Buda [1]. The dataset in turn is the LGG Segmentation Dataset, which focuses on the segmentation of abnormal regions in brain MRI images from patients diagnosed with lower-grade glioma (LGG). This dataset was built from medical imaging data obtained from The Cancer Imaging Archive (TCIA) and is linked to cases from The Cancer Genome Atlas (TCGA) lower-grade glioma collection. In addition to imaging data, the dataset also includes clinical and genomic information for each patient through the file data.csv.

This dataset is designed for the study of brain tumor abnormalities visible in FLAIR MRI sequences, and it is commonly used in medical image segmentation and radiogenomic analysis. The segmentation masks were created manually and serve as the ground truth for supervised learning tasks.

The dataset is especially relevant because it combines two important sources of information:

1. Medical imaging data, which supports the segmentation task.

2. Patient-level genomic and clinical data, which can be used for complementary analysis and future radiogenomic studies.

2.2 Structure of the Dataset

The dataset is organized into **110 folders**, each corresponding to one patient. Every folder contains multiple MRI slices and their corresponding segmentation masks. The files follow a naming convention that allows each MRI image to be matched directly with its mask.

The general naming format is:

TCGA_<institution-code>_<patient-id>_<slice-number>.tif

and the corresponding mask is:

TCGA_<institution-code>_<patient-id>_<slice-number>_mask.tif

This means that for each MRI slice there is an associated mask file with the same identifier and the suffix mask.

2.3 File Types and Format

The images are stored in .tif format and each image has 3 channels. These channels correspond to different MRI sequences:

- pre-contrast
- FLAIR
- post-contrast

For some patients, one of these sequences is missing. In those cases, the missing sequence was replaced with the FLAIR sequence so that all images maintain a consistent 3-channel structure.

The segmentation masks are also stored in .tif format, but they are single-channel binary images. These masks indicate whether each pixel belongs to the abnormal region visible in the FLAIR sequence. In the explored dataset, the mask values are represented as 0 for background and 255 for abnormal region, which can be converted into 0 and 1 for training purposes.

2.4 Number of Patients and Samples (instances)

The dataset contains information for 110 patients. From the exploratory analysis, the dataset includes:

- 3929 MRI images
- 3929 segmentation masks

All MRI images have shape:

- $256 \times 256 \times 3$

All masks have shape:

- 256×256

Thus, the dataset is composed of 2D MRI slices rather than full 3D volumes, and each slice is paired with its corresponding segmentation mask.

2.5 Additional Information

Besides the image and mask files, the dataset includes a file called data.csv, which contains patient-level metadata. This file provides information such as:

- **Genomic subtype clusters:** molecular groupings of tumors based on genomic data, used to identify biologically similar glioma subtypes.
- **Histological type:** tissue-based classification describing the tumor according to its microscopic characteristics.
- **Tumor grade:** indicates the severity or aggressiveness of the tumor.
- **Tumor location:** anatomical region of the brain where the tumor is found.
- **Gender**: biological sex of the patient.
- **Age:** patient age at the time of diagnosis or pathological evaluation.
- **Race:** demographic category reported for the patient.
- **Ethnicity:** ethnic background reported for the patient.
- **Survival-related information:** variables associated with patient outcome, such as survival status or death indicator.

This metadata is not the direct target of the segmentation problem, but it is useful for understanding the patient and supporting complementary analyses.

2.6 Exploration

2.6.1 Average Image

The following image is a recreation of the average image of the dataset for the three channels.

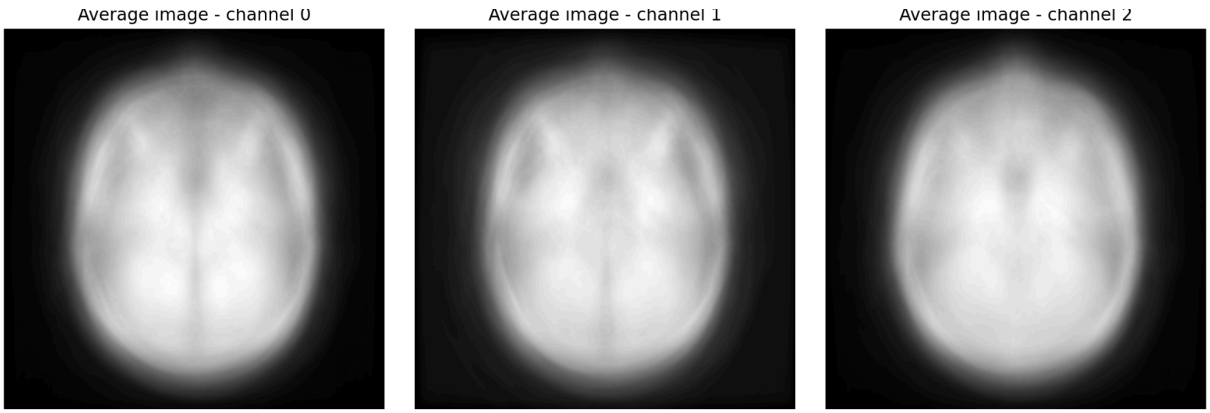


Fig. 1. Average image of the dataset for the three channels.

2.6.2 Class distribution

For the image segmentation task, the labels are provided through the segmentation masks, where each pixel belongs to one of two classes.

- **Class 0:** Represents the background, meaning pixels that do not correspond to the abnormal region and therefore are not the target of interest.
- **Class 1:** Represents the FLAIR abnormality region, which is the area that the model is expected to detect and segment.

Table 1. Pixel class distribution.

Class	Number of pixels	Percentage
0	254,839,090	98.97%
1	2,651,854	1.03%

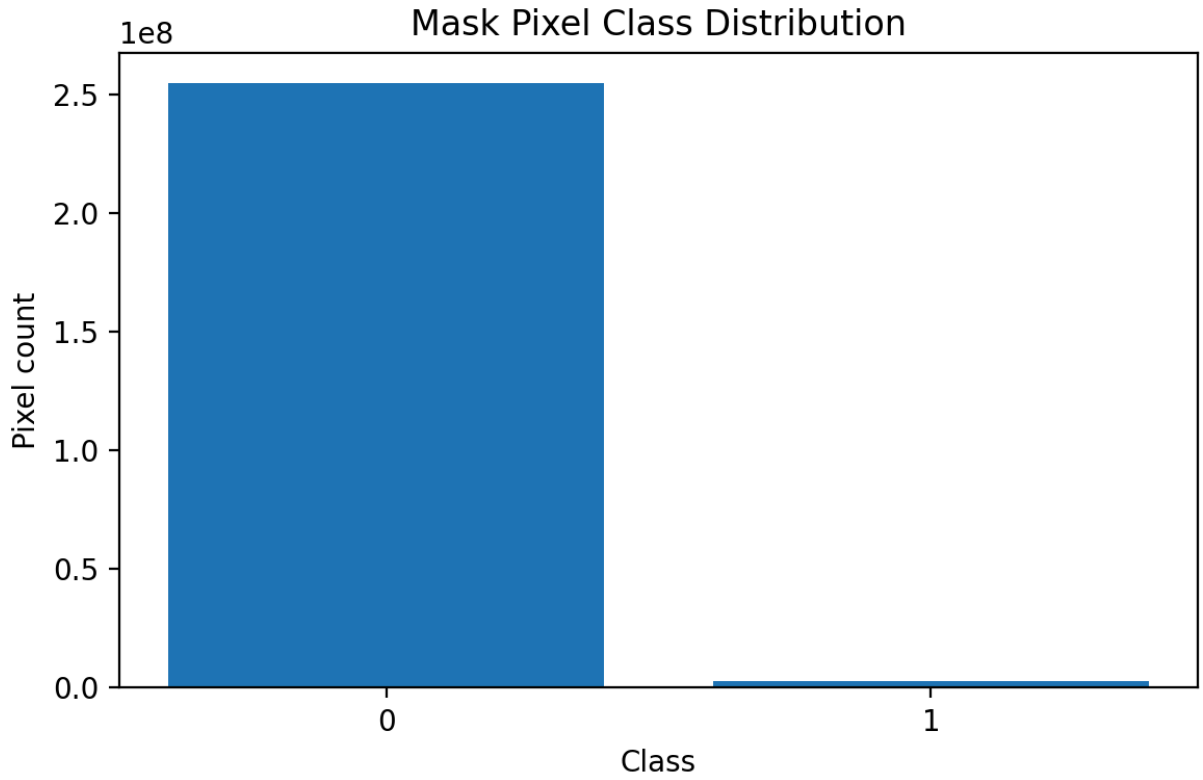


Fig. 2. Mask pixel distribution.

We can note that the dataset is far from being balanced, this due the nature of the dataset and difficult to get new instances.

2.7 Data quality

The quality of the dataset is appropriate for a medical image segmentation task, since it provides a consistent structure across samples and includes manually annotated masks that serve as reliable ground truth. All MRI slices have the same spatial resolution of 256 × 256 pixels, and the images are stored in a uniform 3-channel .tif format, which facilitates preprocessing and model training. In addition, each image has a corresponding segmentation mask, allowing a direct pairing between input data and target labels. However, the dataset also presents some challenges that must be considered during analysis, such as the strong class imbalance.

3.0 Data Preparation

3.1 Data split

To split the dataset, a patient-level strategy was selected in order to avoid data leakage. Since each patient contains multiple MRI slices, all images and masks from the same patient must remain in the same subset.

First, a final test set is reserved using approximately 10–15% of the 110 patients. The remaining patients are then used in a 5-fold cross-validation scheme. In each fold, 4 parts are used for training and 1 part is used for validation. This approach allows the model to be validated several times while keeping an independent final test set for the final evaluation.

Table 2. Data split patient distribution.

Stage	Distribution	Approximate patients	Purpose
Final test set	10–15%	11–17	Final unbiased evaluation
Remaining data	85–90%	93–99	Used for cross-validation
Training in each fold	4/5 of remaining data	about 74–79	Model training
Validation in each fold	1/5 of remaining data	about 19–20	Model validation

3.2 Data augmentation

In order to improve the robustness of our data set, the next data augmentation modifications will be applied for the training data, with

Table 3. Data augmentation.

Data augmentation	Description / purpose
Horizontal flip	Introduces spatial variation in the image orientation and helps the model generalize better.
Small rotations	Simulates slight changes in image orientation during acquisition while preserving the anatomical structure.
Small shifts / translations	Moves the image slightly in horizontal or vertical directions to improve robustness to position changes.
Small scaling / zoom	Helps the model adapt to small variations in the apparent size of the abnormal region.
Brightness adjustment	Simulates differences in image intensity between scans.

Contrast adjustment	Improves robustness to variations in tissue contrast and scanner conditions.

The selected augmentations are not intended to be applied aggressively. Instead, they are used with small magnitudes and controlled probability. In this way, augmentation increases variability in the training data without introducing unrealistic distortions.

3.3 Prepared dataset

After the data augmentation applied to training data and the separation into folds, the ramming processed data follows the next structure:

Table 4: Prepared dataset split.

Stage / Fold	Subset	Patients	Image-mask pairs
Final split	Train + Validation pool	99	3599
	Final Test	11	330
Fold 1	Train	79	2809
	Validation	20	790
Fold 2	Train	79	2784
	Validation	20	815
Fold 3	Train	79	2926
	Validation	20	673
Fold 4	Train	79	2942
	Validation	20	657
Fold 5	Train	80	2935
	Validation	19	664

4.0 Modeling

The modeling phase was divided into N iterations, where each one has a different purpose in the project, since they have a baseline to compare the architecture proposals and the improvement of those depending on the results of each iteration. This approach is pretended to improve the model and get better results.

4.1 Threshold-based segmentation

A segmentation threshold was selected, a very basic classical image processing algorithm or a rule-based heuristic method, that is not pretended to get accurate results, and is a start point for comparison against the next models. Threshold segmentation is an appropriate baseline for image segmentation because it is a simple, interpretable, and computationally inexpensive method that directly separates foreground from background based on pixel intensity values. The disadvantage of this approach is that it does not learn patterns or spatial context from data.

4.1.1 Parameters and configuration

Table 5. Threshold-based segmentation parameters configuration.

Aspect	**Configuration**
Model name	threshold_baseline
Model type	Classical non-trainable baseline
Segmentation approach	Fixed threshold segmentation
Input image type	Multi-channel .tif image
Channel used for segmentation	Channel index 1
Channel name used	FLAIR
Normalization	Min-max normalization per image
Threshold value	0.55
Output	Binary mask
Output classes	0 = background, 1 = lesion
Training required	No
Learned weights	No
Cross-validation folds	5
Validation strategy	Fold-based validation on fold 1 to fold 5
Final evaluation	Separate final test split

4.2 Encoder - decoder v1

An encoder-decoder architecture was selected for a more solid result. This deep learning architecture was chosen because it is specifically designed to solve the two main requirements of the task: understanding the content of the image and recovering the spatial location of each region at pixel level. The encoder progressively transforms the input image into a compact set of high-level feature representations, allowing the model to capture relevant patterns, textures, and contextual information. The decoder then upsamples these learned representations to recover the spatial resolution needed to generate a pixel-wise segmentation mask.

This is a first base mode inspired from state of the art encoder-decoder architectures, already proven with excellent results, architecture such as U-Net and nnU-Net[8][10]. However this is a very basic start point, more features from these architectures are considered to be implemented in next versions of this model, depending on the objective of next iterations.

Despite the advantages, this architecture represents some limitations to consider such as the loss of spatial detail during downsampling, possible coarse boundary reconstruction, higher computational cost, sensitivity to hyperparameters, and risk of overfitting on limited datasets.

4.2.1 Parameters and configuration

Table 6. Parameters configuration for encoder-decoder v1.

Aspect	**Configuration**
Model name	encoder_decoder_v1
Framework	Keras / TensorFlow
Task	Binary image segmentation
Input shape	256 x 256 x 3
Output shape	256 x 256 x 1
Output activation	Sigmoid
Loss function	Binary Cross-Entropy
Optimizer	Adam
Learning rate	1e-3

Batch size	8
Epochs	15
Prediction threshold	0.5
Cross-validation folds	5
Random seed	42
Image normalization	Per-channel min-max normalization
Checkpoint strategy	Best model saved by val_loss
Training time	5.4 ho

4.2.2 Architecture

This first version has a basic configuration with two main encoding and decoding blocks, using convolutional layers followed by ReLU activation functions to extract nonlinear features from the input image. In the encoder, max-pooling layers are applied to progressively reduce the spatial dimensions and capture higher-level contextual information. In the decoder, upsampling layers are used to restore the spatial resolution and reconstruct the segmentation mask. Finally, a 1x1 convolution with sigmoid activation produces a pixel-wise binary output, where each pixel is classified as background or lesion. This configuration represents a basic standard for modern encoder-decoder architectures and leaves room for possible improvements.

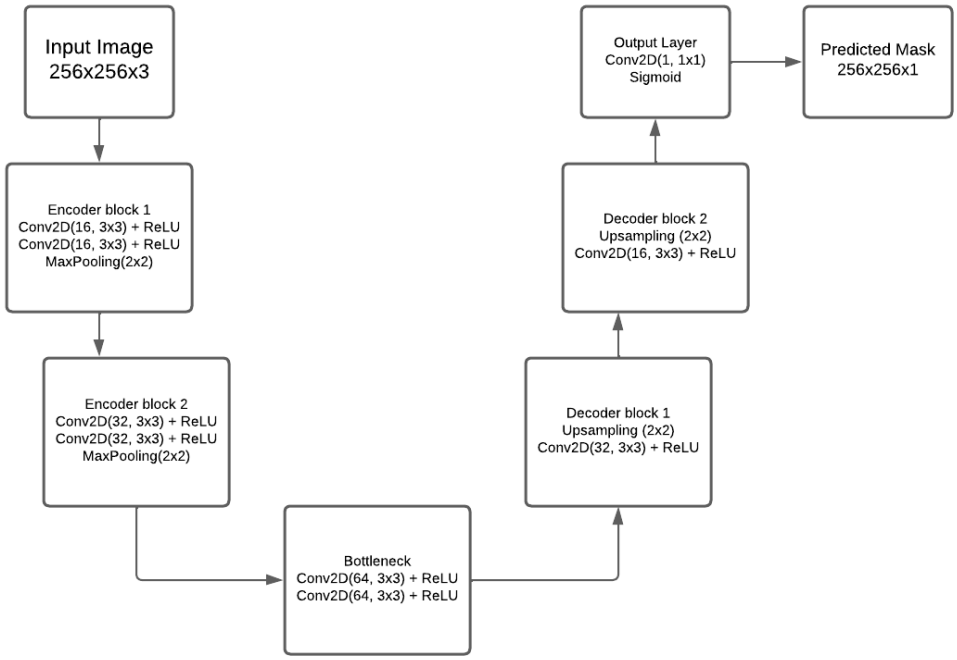


Fig. 3. Encoder-decoder v1 architecture.

This architecture was directly inspired by the image segmentation encoder-decoder model described by Kayasth [5], as well as the U-Net architecture introduced by Ronneberger, Fischer, and Brox [8].

4.3 Encoder - decoder v2

There are two main objectives for this architecture based on the previous version results. The first one is to improve the model through getting a better score for IoU, precision and recall mainly against the version 1; the second one is to make it more computational efficient, this due the long time that takes training the previous architecture, the previous architecture took 6 hours to train. Further are explained all the changes and why that decision was taken.

4.3.1 Changes and improvements

Considering the objectives for this new version of the previous model, the changes applied to the parameters and architecture were strongly inspired by the U-Net architecture proposed by Ronneberger and Fischer. The main new element taken from U-Net is the implementation of skip connections, which allow feature maps from the encoder to be directly transferred to the decoder. This helps preserve spatial information that is usually lost during downsampling and improves the model's ability to recover fine details in the segmented region.

The changes are:

Table 7. Changes and improvements for encoder-decoder v2.

Change	Base model V1	New version (V2)	Why the change was made
Skip connections	No	Yes	To preserve spatial information from the encoder and pass it directly to the decoder, improving localization of the lesion region.
Downsampling method	Max Pooling	Strided convolution	To make downsampling learnable, so the network can learn how to reduce spatial resolution while extracting useful features. This was successfully my previous personal work [9], where adding as tried convolutions instead max pooling can improve the patterns learned. In this case it is thinked to compensate for the filter reduction.
Number of filters	16 → 32 → 64	8 → 16 → 32	To make the model lighter and reduce computational cost while keeping a hierarchical feature representation.
Decoder design	Upsampling + convolution	Upsampling + skip connection + convolution	To improve reconstruction of the output mask by combining decoder features with encoder features.
Encoder design	Convolution blocks + max pooling	Convolution blocks + skip connections + strided convolutions	To preserve more relevant spatial information and make downsampling learnable.
Batch size	8	16	To evaluate whether a larger batch size can improve training speed and computational efficiency if memory allows.
Loss function	Binary Cross Entropy	Dice Loss	To address class imbalance more effectively and optimize overlap between predicted and true lesion regions.

4.3.2 Parameters and configuration

Table 8. Parameters configuration for encoder-decoder v2.

Aspect	Configuration
Model name	encoder_decoder_v2
Framework	Keras / TensorFlow
Task	Binary image segmentation
Input shape	256 x 256 x 3
Output shape	256 x 256 x 1
Output activation	Sigmoid
Loss function	Dice Loss
Optimizer	Adam
Learning rate	1e-3
Batch size	16
Epochs	15
Prediction threshold	0.5
Cross-validation folds	5
Random seed	42
Image normalization	Per-channel min-max normalization
Checkpoint strategy	Best model saved by val_dice_coefficient

4.3.3 Architecture

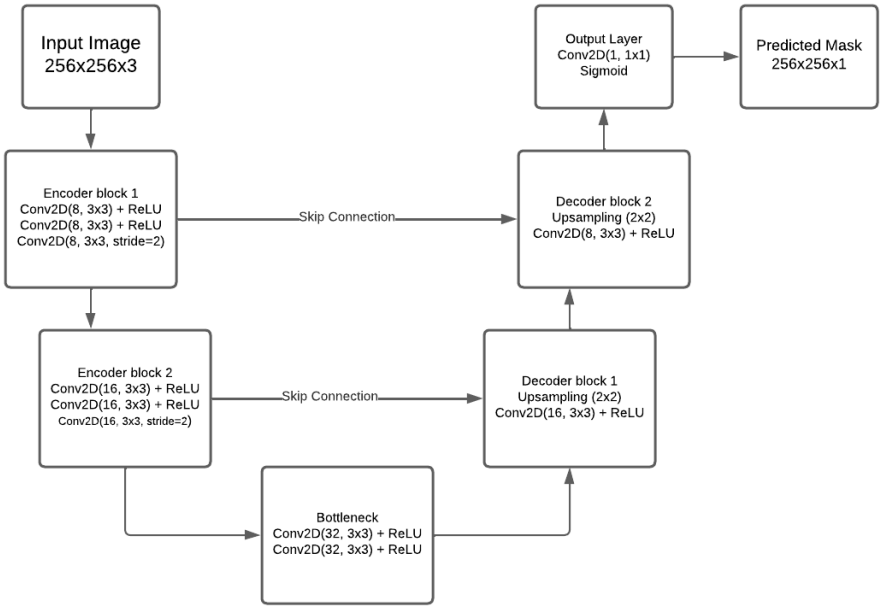


Fig. 4. Encoder-decoder v2 architecture.

4.4 Encoder - decoder v3

Due to the failure of the previous version, this new one keeps the same U-Net like architecture as the previous one but with some changes to the architecture and parameters. The objective of this version is not to have a more cost-efficient model, this could be addressed in future models; The objective gets a real improvement against the metrics of the version 1.0.

4.4.1 Changes and improvements

Table 9. Changes and improvements for encoder-decoder v2.

Change	Previous version (V2)	New version (V3)	Why the change was made
Number of filters	8 → 16 → 32	16 → 32 → 64	To recover representational capacity, since the reduced filters in V2 likely made the model too weak to learn foreground lesion patterns robustly.

Decoder capacity	16 → 8	32 → 16	To make the decoder consistent with the stronger encoder and improve reconstruction of lesion regions in the output mask.
Decoder convolution depth	One convolution after each skip concatenation	Two convolutions after each skip concatenation	To strengthen feature refinement in the decoder after merging encoder and decoder information.
Loss function	Dice loss	BCE + Dice loss	To improve optimization stability, since pure Dice loss may favor unstable training or collapse in highly imbalanced segmentation settings.
Batch size	16	8	To make updates less coarse and closer to the baseline setting, potentially improving learning stability.
Prediction threshold strategy	0.5 only	0.5 as default + sweep of 0.5, 0.3, 0.25, 0.2	To evaluate whether the model predicts useful foreground probabilities below 0.5 and select a better operating threshold.

4.4.2 Parameters and configuration

Table 10. Parameters configuration for encoder-decoder v3.

Aspect	Configuration
Model name	encoder_decoder_v3
Framework	Keras / TensorFlow
Task	Binary image segmentation
Input shape	256 x 256 x 3
Output shape	256 x 256 x 1
Output activation	Sigmoid
Loss function	BCE Dice Loss

Optimizer	Adam
Learning rate	1e-3
Batch size	8
Epochs	15
Prediction threshold	0.5
Cross-validation folds	5
Random seed	42
Image normalization	Per-channel min-max normalization
Checkpoint strategy	Best model saved by val_dice_coefficient

4.4.3 Architecture

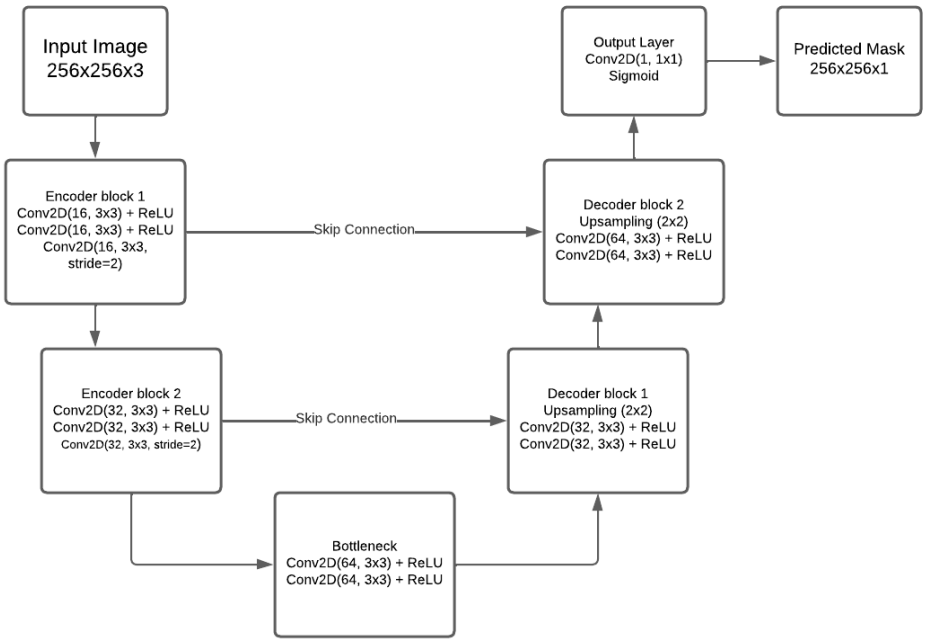


Fig. 5. Encoder-decoder v3 architecture.

4.5 Encoder - decoder v4

This final iteration of the model, try to resolve the following problems:

- V1 has better metrics, but visually produces more false positives. That means its recall/IoU may look good, but it is probably over-segmenting.
- V3 is more conservative, so even if its metrics are lower, it may be closer to the real clinical-looking mask because it avoids predicting lesions where there is none.

Based on that, this version takes the best features of both models in order to keep V3's conservative behavior while recovering V1's efficiency.

4.5.1 Changes and improvements

Table 11. Changes and improvements encoder-decoder v4.

Change	Base model V3	New version V4	Why the change was made
Convolution type	Standard Conv2D	SeparableConv2D	To reduce computational cost while keeping the same general convolutional structure. Separable convolutions require fewer operations than standard convolutions, making training faster.
Downsampling method	Strided convolution	Max Pooling	To simplify the downsampling process and reduce training cost. Max Pooling is cheaper than using an additional learnable convolution for downsampling.
Number of encoder blocks	2 encoder blocks	3 encoder blocks	To add one more spatial level and allow the model to learn features at a deeper resolution while keeping the model efficient.
Number of filters	16 → 32 → 64	8 → 16 → 32 → 64	To reduce the cost of the early layers while still reaching the same bottleneck capacity of 64 filters.
Decoder design	32 → 16	32 → 16 → 8	To reconstruct the mask progressively across three decoder stages instead of two, matching the deeper encoder structure.
Learning rate strategy	Fixed learning rate	ReduceLROnPlateau	To reduce the learning rate when validation performance stops improving, allowing more stable training.

4.5.2 Parameters and configuration

Table 12. Parameters and configuration encoder-decoder v4.

Aspect	Configuration
Model name	encoder_decoder_v4
Framework	Keras / TensorFlow
Task	Binary image segmentation
Input shape	256 x 256 x 3
Output shape	256 x 256 x 1
Output activation	Sigmoid
Loss function	BCE + Dice Loss
Optimizer	Adam
Learning rate	1e-3
Batch size	8
Epochs	15
Prediction threshold	0.5
Cross-validation folds	5
Random seed	42
Image normalization	Per-channel min-max normalization
Checkpoint strategy	Best model saved by val_dice_coefficient

4.5.3 Architecture

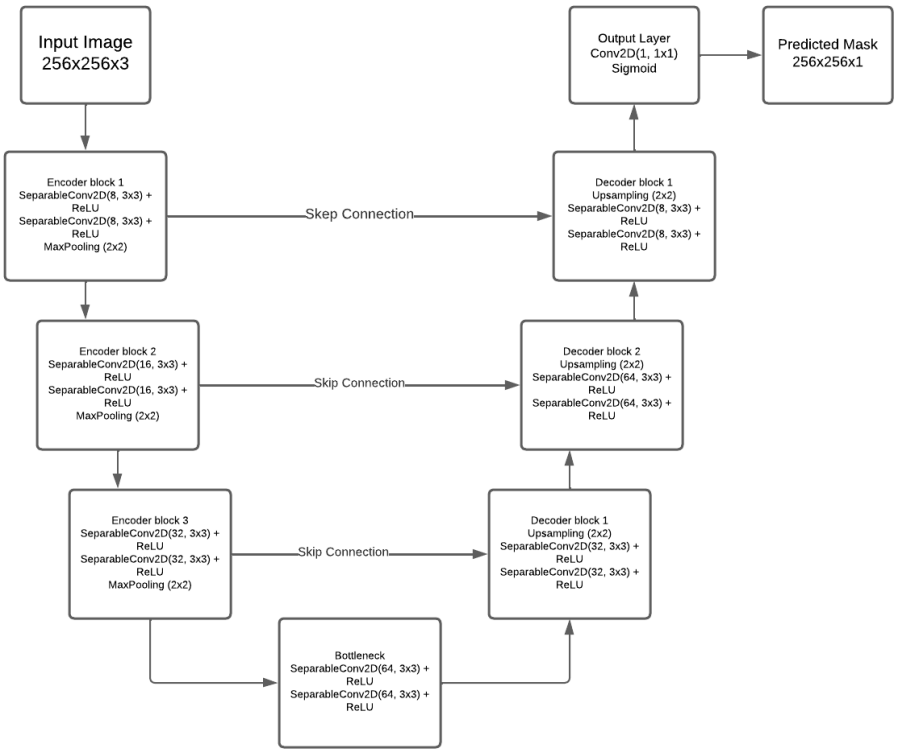


Fig. 6. Architecture encoder-decoder v4.

5.0 Evaluation

The evaluation section aims to measure the segmentation performance of each model in a consistent and comparable way. Since all approaches in this study produce a predicted segmentation mask, the same evaluation metrics can be applied across classical and deep learning models. To assess performance, the metrics selected were Intersection over Union (IoU), Precision, Recall, and Pixel Accuracy.

- **IoU:** The primary metric because it directly measures the agreement between predicted and ground-truth masks.
- **Recall:** Also highly relevant, especially in lesion segmentation, where missing true lesion pixels is undesirable.
- **Precision:** Helps identify over-segmentation.
- **Pixel Accuracy:** Reported only as a complementary metric due to the strong class imbalance.

5.1 Threshold-based segmentation

5.1.1 Results

Table 13. Threshold-based segmentation metrics results.

Fold / Split	TP	FP	TN	FN	IoU	Precision	Recall	Pixel Accuracy
Fold 1	122,657	1,611,309	49,589,050	450,424	0.0562	0.0707	0.2140	0.9602
Fold 2	88,692	1,816,349	51,072,561	434,238	0.0379	0.0466	0.1696	0.9579
Fold 3	53,812	1,442,680	42,344,552	264,684	0.0306	0.0360	0.1690	0.9613
Fold 4	127,653	1,705,901	40,820,137	403,461	0.0571	0.0696	0.2404	0.9510
Fold 5	127,238	1,872,837	41,166,491	349,338	0.0542	0.0636	0.2670	0.9489
Global CV	520,052	8,449,076	224,992,791	1,902,145	0.0478	0.0580	0.2147	0.9561
Final Test	61,434	681,908	20,715,315	168,223	0.0674	0.0826	0.2675	0.9607

5.1.2 Metrics analysis

- **IoU:** This is the most important metric in this baseline analysis because it directly measures overlap between the predicted mask and the ground truth. The obtained values are very low, which indicates poor segmentation quality.

- **Precision:** Precision is also very low, showing that a large proportion of the pixels predicted as lesions actually correspond to background. This suggests many false positives.

- **Recall:** Recall is slightly higher than Precision, meaning that the method is able to detect part of the lesion area, but still misses a large amount of true lesion pixels.

- **Pixel Accuracy:** Although Pixel Accuracy is high, it must be interpreted carefully because the dataset contains a large majority of background pixels. For that reason, this metric is less informative than IoU for this task.

5.1.3 Folds and test analysis

Overall, the behavior across folds suggests that thresholding is unstable and limited for this problem, since performance changes between folds but remains weak in all cases.

The final test slightly improved over global cross-validation, but performance remained low overall. This confirms that thresholding can serve as a basic reference baseline, yet it is not strong enough for accurate lesion segmentation due to its limited overlap and low precision.

5.1.4 Conclusions

These results confirm that threshold segmentation is useful as a simple baseline, but its low overlap and low precision demonstrate that more advanced learning-based models are necessary for this problem.

5.2 Encoder - decoder V1

5.2.1 Results

Table 14. Encoder-decoder v1 segmentation metrics results.

Fold / Split	TP	FP	TN	FN	IoU	Precision	Recall	Pixel Accuracy
Fold 1	255,563	129,210	51,071,149	317,518	0.3639	0.6642	0.4459	0.9914
Fold 2	313,217	75,903	52,813,007	209,713	0.5230	0.8049	0.5990	0.9947
Fold 3	218,421	79,043	43,708,189	100,075	0.5494	0.7343	0.6858	0.9959
Fold 4	357,662	103,124	42,422,914	173,452	0.5639	0.7762	0.6734	0.9936
Fold 5	225,841	22,261	43,017,067	250,735	0.4527	0.9103	0.4739	0.9937
Global CV	1,370,704	409,541	233,032,326	1,051,493	0.4841	0.7700	0.5659	0.9938
Final Test	175,608	80,671	21,316,552	54,049	0.5659	0.6852	0.7647	0.9938

5.2.2 Metrics analysis

- **IoU:** IoU shows a clear improvement compared with the threshold baseline, indicating that the encoder-decoder achieves a much stronger overlap between predicted masks and ground truth.

- **Precision:** Precision is high in most folds, which means that a large proportion of the predicted lesion pixels are correct.

- **Recall:** Recall is also considerably higher than in the threshold baseline, showing that the model is able to recover a larger portion of the true lesion area.

- **Pixel Accuracy:** Pixel Accuracy remains very high, but as in the previous case, it should be interpreted carefully because background pixels dominate the images.

5.2.3 Folds and test analysis

The encoder-decoder shows substantially better and more consistent performance than the threshold baseline across all folds. Folds 3 and 4 achieved the strongest balance between Precision and Recall, resulting in the highest IoU values. Overall, the fold results show that the encoder-decoder is capable of learning meaningful lesion patterns, although some variability between folds is still present.

The combination of a higher Recall and solid IoU in test suggests that the model produced a more complete lesion segmentation on the final test set. This confirms that the encoder-decoder provides a much more effective segmentation approach than the threshold baseline and establishes a solid first deep learning benchmark.

5.2.4 Conclusions

These results show that the encoder-decoder significantly outperforms the threshold baseline and provides a strong initial deep learning solution for lesion segmentation. However this model can get a better performance and according to the previous results the work to be done in order to increase the performance of our model but not significantly considering the computational cost.

5.3 Encoder - decoder V2

5.3.1 Results

Training was stopped because the model showed clear evidence of early collapse to background prediction. After the first epoch, both training and validation metrics became completely flat, which indicates that the optimization process was no longer producing meaningful learning. In addition, the per-image validation results showed TP = 0 and FP = 0 across the reported samples, meaning that the model predicted only background pixels. For slices with actual lesion regions, this caused FN > 0, IoU = 0, Precision = 0, and Recall = 0, which confirms that the model was not segmenting the foreground class.

5.3.4 Conclusions

Based on this behavior, continuing the same training was not justified, since the model had already converged to a poor trivial solution and was not improving across epochs.

Considering this, the main objective is back to a configuration that gets better performance sacrificing or leaving for further models or versions the optimization of the model. A proposed decisions is:

Table 15. Encoder-decoder v2 improvements proposal.

Improvement	Reason
Restore filter sizes closer to V1	To recover model capacity and improve feature extraction

Keep skip connections	To preserve spatial information and support localization
Test lower thresholds such as 0.4 or 0.3	To avoid discarding weak but useful foreground probabilities
Consider BCE + Dice loss	To improve optimization stability and reduce collapse to all-background predictions

5.4 Encoder - decoder V3

5.4.1 Results

Table 16. Encoder-decoder v3 segmentation metrics results.

fold	TP	FP	TN	FN	IoU	Precision	Recall	Pixel Accuracy
fold_1	309828	716274	50484085	263253	0.240297	0.301947	0.540636	0.981081
fold_2	126591	43692	52845218	396339	0.223413	0.743415	0.242080	0.991762
fold_3	109565	174853	43612379	208931	0.222084	0.385225	0.344007	0.991299
fold_4	176831	50009	42476029	354283	0.304292	0.779541	0.332944	0.990610
fold_5	222131	81333	42957995	254445	0.398149	0.731985	0.466098	0.992284
global CV	944946	1066161	232375706	1477251	0.270886	0.469864	0.390119	0.989217
Final test	38053	53730	17975880	151345	0.156514	0.414597	0.200916	0.988744

5.4.2 Metrics analysis

- **IoU:** The values are considerably lower than those obtained with the encoder-decoder V1. Although the model is able to detect lesion regions, the overlap between predicted masks and

ground truth remains limited in most folds. The global cross-validation IoU of 0.270886 and the final test IoU of 0.156514 indicate that this version did not achieve a strong segmentation quality.

- **Precision:** Precision is highly variable across folds. In folds 2, 4, and 5, the model achieved relatively high precision, which means that when it predicts lesion pixels, many of them are correct. However, fold 1 shows a very low precision due to a high number of false positives, indicating probably that the model tends to over-segment in some cases.

- **Recall:** Recall is generally low to moderate. Fold 1 obtained the highest Recall, but this came together with a very large number of false positives. In the other folds, Recall remained limited, showing that the model is still missing a substantial portion of the true lesion area. This issue becomes more evident in the final test, where Recall drops to 0.200916, suggesting that the model fails to recover much of the lesion region on unseen data.

- **Pixel Accuracy:** As in the previous models, this metric must be interpreted carefully because background pixels dominate the images.

5.4.3 Fold and test analysis

The results show that this version presents a high variability across folds and does not maintain a stable balance between Precision and Recall. Fold 1 obtained the highest Recall, but its very low Precision indicates strong over-segmentation. In contrast, folds 2 and 4 reached high Precision values, but their Recall remained low, meaning that the model produced conservative predictions and missed many lesion pixels. Fold 5 achieved the best overall balance among the validation folds, resulting in the highest IoU value of 0.398149, although it is still below the values reached by the previous version 1.

5.4.4 Conclusions

The results did not achieve the expected results and the goal, against all the waited adjustments for a better performance, taking a strong inspiration from a U-Net architecture, the results given are worse than the most basic version and the time that takes to train this model was about 10 hours, four more than the version 1.

Although the results are worse, this is not necessarily something bad, this model could be more conservative and we need to consider the unbalance of classes, and this model could lead to more true positives and less false positives.

5.5 Encoder - decoder V4

5.4.1 Results

Table 17. Encoder-decoder v4 segmentation metrics results.

fold	Threshold	TP	FP	TN	FN	IoU	Precision	Recall	Pixel Accuracy
fold_1	0.5	391341	164241	5103611 8	181740	0.530760	0.704380	0.682872	0.993317
fold_2	0.5	384863	331292	5255761 8	138067	0.450542	0.537402	0.735974	0.991212
fold_3	0.5	248286	164080	4362315 2	70210	0.514501	0.602101	0.779558	0.994688
fold_4	0.5	418243	134907	4239113 1	112871	0.627973	0.756111	0.787483	0.994245
fold_5	0.5	335987	201483	4283784 5	140589	0.495513	0.625127	0.705002	0.992139
global_cv	0.5	1778720	996003	2324458 64	643477	0.520367	0.641044	0.734342	0.993049

5.5.2 Metrics Analysis

- **IoU:** The results show a clear improvement in overlap quality compared with the weaker previous versions. Most validation folds achieved IoU values around 0.45 to 0.63, with fold 4 reaching the best validation IoU of 0.627973, the best across all models. indicates that this version is able to produce a much better agreement between predicted masks and ground truth.

- **Precision:** Precision is overall solid, although it still varies across folds. This means that in most cases the model is predicting lesion pixels with a good level of correctness, but some folds still show a moderate number of false positives.

- **Recall:** Recall is consistently strong across all the model versions. This indicates that the model is able to recover a large portion of the true lesion area.

- **Pixel Accuracy:** As before this metrics is not reliable due the high composition of background of the images and unbalanced dataset.

5.5.3 Folds and test analysis

Overall, the fold results show that this version is much more stable and effective than the previous ones. Although there is some variability between folds, most of them achieved good overlap and a balanced relationship between Precision and Recall, with fold 4 standing out as the strongest validation result. The final test performance is also strong, confirming that the model generalizes well to unseen data. In general, this version demonstrates a clear improvement in segmentation quality and provides more reliable results across different patient partitions.

5.5.4 Conclusión

This model can be considered the best version due to its strong performance and the consistency of its results across the different folds. In addition to achieving the highest segmentation quality overall, it was trained in approximately 3 hours, which makes it a cost-efficient solution in terms of computational time. Therefore, this version successfully meets the objectives established for version 3, as it provides a good balance between performance, stability, and efficiency.

6.0 Model comparison

Next will be shown an average of the matrices across the folds of each model to compare directly the results.

Table 18. Models metrics comparison

Model	Training time	IoU	Precission	Recall	Accuracy
Threshold-based segmentation	< 1 minute	0.0478	0.0580	0.2147	0.9561
Encoder-decoder V1	6.25 hours	0.5659	0.6852	0.7647	0.9938
Encoder-decoder V2	N/A	N/A	N/A	N/A	N/A
Encoder-decoder V3	10 hours	0.156514	0.414597	0.200916	0.988744
Encoder-decoder V4	3.12 hours	0.520367	0.641044	0.734342	0.993049

We can note that the version 1 and version 4 or the model had very similar results, being version 1 slightly better, but the training time in the version 4 made it more efficient by a reduction of half of the training time. Also both versions V1 and V2 had better results in each metric, but the version 3 played a key role in the improvement of itself and with features of the V1 that resulted in the best model overall. Threshold

got very poor results but thought was necessary as a baseline because sometimes the best solution is the simplest way, unfortunately this is not the case for threshold.

Best metrics:

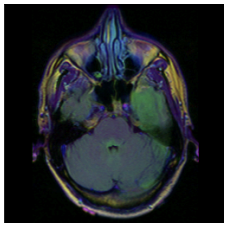
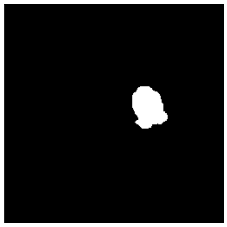
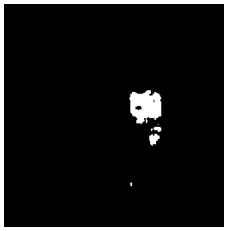
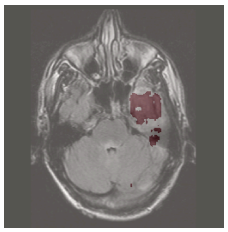
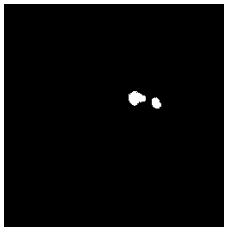
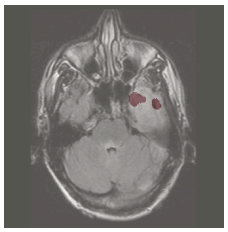
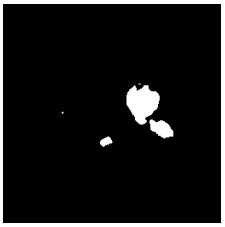
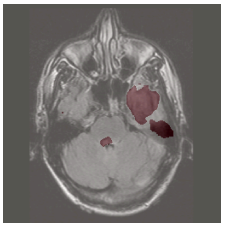
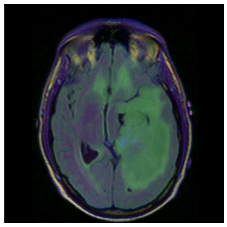
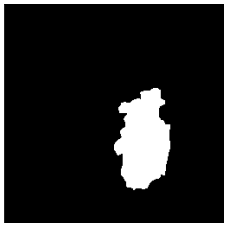
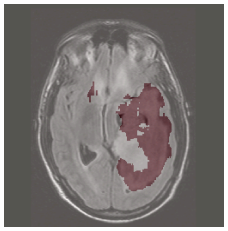
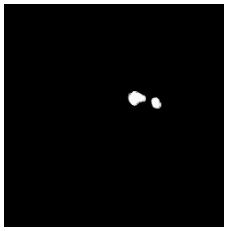
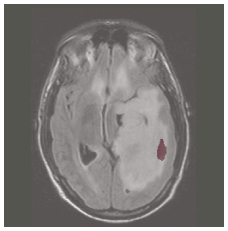
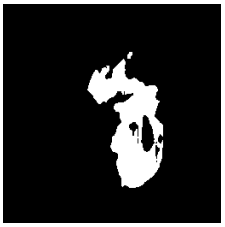
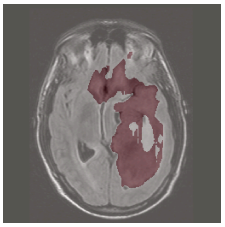
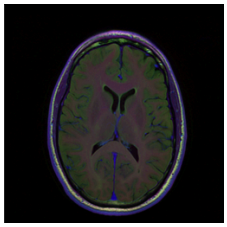
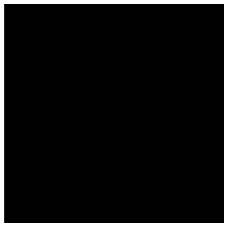
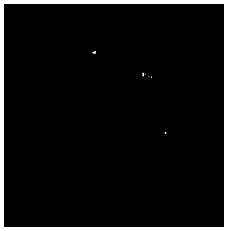
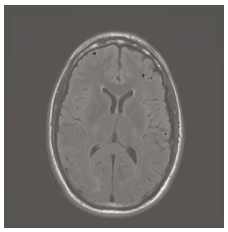
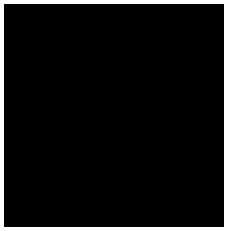
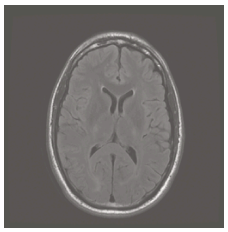
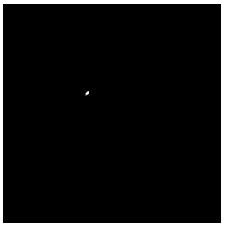
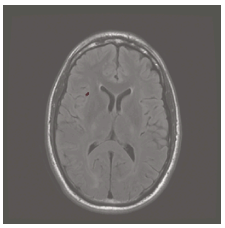
- **IoU:** Version 4, fold 4.
- **Precision:** Version 1, fold 5.
- **Recall:** Version 4, fold 4.

7.0 Predictions

Three images were selected for the prediction, in the next table will be shown the results of each model with a threshold of 0.5.

Note: Threshold-based and encoder-decoder v2 models were discarded for this comparison due the overwhelming performance differences against encoder-decoder models version 1, 3 and 4.

Table 19. Image mask predictions comparison

Image	Mask	Encoder-decoder V1	Encoder-decoder V3	Encoder-decoder V4
		 	 	 
			 	 
		 	 	 

In terms of predictions the best model is the first version, followed by the version 4. This both throw similar results but the version predicts more false positives than the frist version. This could be fixed with the adjustment of the parameters of the model such as the threshold. In some cases like the third image the

version 4 is more conservative than the version one but in other cases such as the second image, the opposite. In any case, justifying the version 1 over the version 4 just by a very small difference is not worth it, considering that the version 4 is way lighter and as mentioned this could be improved by changing some parameter or even the architecture taking version 4 as backbone.

8.0 General Conclusions

The results show that the encoder-decoder version 4 model is the best overall solution developed in this project. Although encoder-decoder version 1 obtained slightly better metrics in some cases, V4 achieved a better balance between performance, stability, and computational efficiency. The model reached a global cross-validation IoU of 0.520367 and a final test IoU of 0.600055, while also obtaining strong Recall and Pixel Accuracy.

The main advantage of V4 is that it significantly reduced training (with a training time reduction of 49%) time while maintaining strong segmentation quality. By using separable convolutions, MaxPooling, skip connections, and a lightweight deeper encoder-decoder structure, the model became more efficient without losing the ability to detect lesion regions. This makes version 4 a practical and cost-efficient model for this segmentation task.

Overall, this project demonstrates that a lightweight encoder-decoder architecture can achieve competitive results for brain MRI segmentation. Future work could focus on threshold optimization, additional post-processing to reduce false positives, and further architectural improvements using version 4 as the backbone model.

References

[1] M. Buda, "Brain MRI segmentation," *Kaggle*, [Online]. Available: https://www.kaggle.com/datasets/mateuszbuda/lgg-mri-segmentation/data. [Accessed: Apr. 12, 2026].

[2] K. Alomar, H. I. Aysel, and X. Cai, "Data augmentation in classification and segmentation: A survey and new strategies," *Journal of Imaging*, vol. 9, no. 2, p. 46, 2023. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC9966095/. [Accessed: Apr. 12, 2026].

[3] GeeksforGeeks, "Thresholding-Based Image Segmentation," *GeeksforGeeks*. [Online]. Available: https://www.geeksforgeeks.org/software-engineering/thresholding-based-image-segmentation/. [Accessed: Apr. 15, 2026].

[4] GeeksforGeeks, "SegNet - A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation," *GeeksforGeeks*. [Online]. Available: https://www.geeksforgeeks.org/computer-vision/segnet-a-deep-convolutional-encoder-decoder-architecture-for-image-segmentation/. [Accessed: Apr. 17, 2026].

[5] J. Kayasth, "Image Segmentation using Encoder-Decoder model," preprint, Dec. 2022. [Online]. Available: ResearchGate. [Accessed: Apr. 20, 2026.]

[6] V. Badrinarayanan, A. Kendall, and R. Cipolla, "SegNet: A deep convolutional encoder-decoder architecture for image segmentation," *arXiv preprint* arXiv:1511.00561, Oct. 2016.

[7] G. Xu, X. Wang, X. Wu, X. Leng, and Y. Xu, "Development of skip connection in deep neural networks for computer vision and medical image analysis: A survey," *arXiv preprint* arXiv:2405.01725, May 2024. [Accessed: Apr. 20, 2026.]

[8] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," *arXiv preprint* arXiv:1505.04597, May 2015. [Accessed: Apr. 20, 2026.]

[9] J. Ramirez, "Multi-Label Image Classification of Pokemon Types Using Deep Learning" in *Reto-ML*, GitHub repository. [Online]. Available: https://github.com/j1rd2/Reto-ML/blob/main/report.pdf. Accessed: Apr. 20, 2026. [Accessed: Apr. 20, 2026.].

[10] F. Isensee, J. Petersen, A. Klein, D. Zimmerer, P. F. Jaeger, S. Kohl, J. Wasserthal, G. Koehler, T. Norajitra, S. Wirkert, and K. H. Maier-Hein, "nnU-Net: Self-adapting framework for U-Net-based medical image segmentation," *arXiv preprint* arXiv:1809.10486, Sep. 2018. doi: 10.48550/arXiv.1809.10486. [Accessed: Apr. 20, 2026.].